Secrets in Azure Databricks: scopes, Key Vault and the patterns to avoid

Pipelines need passwords, keys and tokens to reach other systems. Secrets management keeps those credentials out of code, controls who can read them, and lets you rotate them in one place.

> LEARNING OBJECTIVE
>
> Describe secret scopes, Azure Key Vault integration and credential anti-patterns.

01

Why: what goes wrong without secrets management

Why do we need secrets management?

Pipelines connect to databases, APIs and storage with passwords, keys and tokens. A credential typed into code is copied everywhere the code goes, and anyone who reads it can become that pipeline.

RISK	PROTECTION
Password pasted into a notebook, readable by every viewer and kept in revision history	**Secret scope.** Code holds only a name; the value is fetched at run time.
Code with a credential is committed to Git and copied to every clone	**Key Vault.** The secret never enters the repository.
Credential appears in job output, logs or cluster settings	**Redaction + ACLs.** Output shows [REDACTED]; only granted principals can read.
Rotating means editing every notebook, so nobody rotates	**One source of truth.** Rotate once in Key Vault; every job picks up the new value.

02 Secret scopes and Azure Key Vault

A **secret scope** is a named collection of secrets inside a Databricks workspace. Code asks for a secret by *scope + key name*; the value is fetched at run time and never written into the notebook. Access to each scope is controlled with ACLs.

THE FLOW

1 · AZURE KEY VAULT

Secret is stored

Security team adds `sql-erp-password` to `kv-data-prod`.

2 · SECRET SCOPE

Vault is linked

Scope `kv-data-prod` points at the vault. No secret is copied.

3 · JOB OR NOTEBOOK

Fetched at run time

The job, running as `sp-etl-prod`, has READ on the scope and calls `dbutils.secrets.get`.

4 · OUTPUT

Value is redacted

If printed, the notebook shows `[REDACTED]`. Rotate in Key Vault; the next run uses the new value.

```
# Python
pw = dbutils.secrets.get(scope="kv-data-prod", key="sql-erp-password")
print(pw)                    # [REDACTED]

# Cluster Spark config or environment variable: reference, not value
spark.erp.password {{secrets/kv-data-prod/sql-erp-password}}

-- SQL
SELECT secret('kv-data-prod', 'sql-erp-password');
```

Azure Key Vault-backed scope

ENTERPRISE DEFAULT

A read-only link from Databricks to an existing Key Vault. Secrets are created, rotated and audited in Azure.

> **Setup:** create in the workspace UI (#secrets/createScope) or CLI with the vault's DNS name and resource ID. Requires a user's Entra ID token.

> **Vault access:** grant AzureDatabricks the Key Vault Secrets User role, or Get and List in an access policy.

> **Read-only in Databricks:** add and rotate secrets in Key Vault; Databricks cannot write them.

> **Audit:** every read appears in Key Vault logs as well as Databricks audit logs.

> **Network:** if the vault firewall is on, allow trusted Microsoft services.

Databricks-backed scope

Secrets stored by Databricks in the control plane, managed with the CLI or API.

> **Setup:** databricks secrets create-scope, then put-secret.

> **Storage:** encrypted in the control plane; covered by the managed services CMK if enabled.

> **Use for:** quick starts, or when no Key Vault is available to the team.

> **Limitation:** secrets are managed per workspace, outside your central Azure rotation and audit tooling.

SCOPE ACLS

READ	Read secrets in the scope. Give to the job's service principal group.
WRITE	Read, add and change secrets (Databricks-backed scopes only).
MANAGE	Everything above plus change the scope's ACLs. Platform admins only.

ONE SCOPE PER APP AND ENVIRONMENT

kv-erp-prod	grp-prod-pipelines	READ
kv-erp-dev	grp-data-engineers	READ
all scopes	grp-platform-admins	MANAGE

PREFERENCE ORDER · THE BEST SECRET IS NO SECRET

1 **No secret: managed identity**
Unity Catalog reaches ADLS through the Access Connector's managed identity. Nothing to store or rotate.

2 **No stored secret: federation / OAuth**
CI/CD signs in as a service principal with a federated credential; Databricks issues short-lived OAuth tokens.

3 **Secret in Key Vault via a scope**
For third-party systems that only accept passwords or API keys: an ERP database, an SFTP partner.

× **Secret in code, config or parameters**
Never. See the anti-patterns below.

03 Credential anti-patterns

Each pattern with the problem and the fix.

Hardcoded password

Anyone who can open the notebook reads the password. It stays in revision history and every Git commit even after you delete the line.

× ANTI-PATTERN

```
jdbc_url = "jdbc:sqlserver://erp.database.windows.net"
password = "Summer2026!"
df = spark.read.jdbc(jdbc_url, "sales",
    properties={"user": "etl", "password": password})
```

✓ DO THIS INSTEAD

```
password = dbutils.secrets.get(
    scope="kv-data-prod", key="sql-erp-password")
df = spark.read.jdbc(jdbc_url, "sales",
    properties={"user": "etl", "password": password})
```

Fix: Move the value to Key Vault, reference it through a scope, and rotate the exposed password immediately.

Storage account key in Spark config

An account key grants full control of every container in the account, bypasses Unity Catalog, and is visible to anyone who can view the cluster.

× ANTI-PATTERN

```
# Cluster > Spark config
fs.azure.account.key.stdatalake.dfs.core.windows.net
  kJ8s2...Qw==
```

✓ DO THIS INSTEAD

```
-- No key at all: Unity Catalog + managed identity
CREATE EXTERNAL LOCATION raw_landing
  URL 'abfss://raw@stdatalake.dfs.core.windows.net/'
  WITH (STORAGE CREDENTIAL sc_datalake);
```

Fix: Use a storage credential backed by the Access Connector managed identity. Disable shared key access on the storage account.

Working around redaction

Redaction only matches the exact value. Printing it character by character or encoded bypasses it. Redaction is a safety net against accidents, not a security boundary.

× ANTI-PATTERN

```
s = dbutils.secrets.get("kv-data-prod", "api-key")
print(" ".join(s))       # a p i - k e y ...
print(s.encode().hex())
```

✓ DO THIS INSTEAD

```
# Grant READ only to the identity that needs it
databricks secrets put-acl kv-data-prod \
  grp-prod-pipelines READ
```

Fix: Treat anyone with READ on a scope as able to see the secret. Grant READ narrowly, and give interactive users no access to production scopes.

Personal token in automation

A job or CI pipeline using a person's PAT acts with all of their rights, breaks when they leave, and makes the audit trail point at the wrong person.

× ANTI-PATTERN

```
# CI pipeline variables
DATABRICKS_HOST=https://adb-123.azuredatabricks.net
DATABRICKS_TOKEN=dapi4f2e...   # priya's PAT
```

✓ DO THIS INSTEAD

```
# CI authenticates as a service principal
# via workload identity federation, no stored secret
DATABRICKS_HOST=https://adb-123.azuredatabricks.net
ARM_CLIENT_ID=<sp-etl-prod app ID>
DATABRICKS_AUTH_TYPE=github-oidc
```

Fix: Run automation as a service principal using OAuth with a federated credential. Restrict or disable PATs for users with workspace settings.

One scope for everything

Dev and prod credentials in one scope with broad MANAGE means any user can read production passwords or grant themselves access.

× ANTI-PATTERN

```
scope: shared-secrets
  prod-sql-password, dev-sql-password,
  salesforce-token, sftp-key ...
ACL: users  MANAGE
```

✓ DO THIS INSTEAD

```
scope: kv-erp-prod    grp-prod-pipelines  READ
scope: kv-erp-dev     grp-data-engineers  READ
scope: kv-crm-prod    grp-prod-pipelines  READ
# MANAGE: grp-platform-admins only
```

Fix: Create one scope per application and environment, backed by separate Key Vaults. Grant READ to groups, MANAGE to administrators only.

Secrets as parameters

Widget values and job parameters are stored in plain text in run history and visible to anyone who can view the run.

× ANTI-PATTERN

```
# Job task parameters
{"sftp_password": "Xk29!pq"}

pw = dbutils.widgets.get("sftp_password")
```

✓ DO THIS INSTEAD

```
# Pass the reference, not the value
{"secret_scope": "kv-sftp-prod",
 "secret_key":   "partner-password"}

pw = dbutils.secrets.get(
  dbutils.widgets.get("secret_scope"),
  dbutils.widgets.get("secret_key"))
```

Fix: Pass scope and key names as parameters. The value is resolved inside the run and redacted.

Reference secrets, never paste them

Remove the secret with managed identities where you can. Where you can't, keep it in Key Vault and reference it through a scope.

READ means can see

Redaction prevents accidents only. Grant READ on a scope to the groups whose jobs need it, and nobody else.

Automation runs as a service principal

Never a person's token. Federated credentials and OAuth remove the stored secret from CI/CD entirely.